

AI501/DS843 Mockup Exam

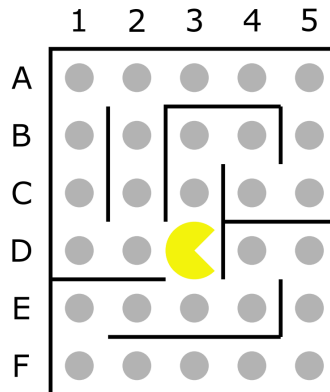
June 202X

Exercises - Open Questions

1 Friendly Yellow Guy

Friendly Yellow Guy lives in the grid environment shown in the figure below. At each time step, he can move from his current position to any adjacent position, as long as there is no wall between them. His purpose in life is to eat all the energy pills in his environment – in particular, if he moves to a position where there is a pill, he will always eat it.

Your task is to help Friendly Yellow Guy to complete his task in the shortest number of moves.



- (a) Classify this problem according to the following dimensions: single/multi-agent; fully/partially observable; deterministic/stochastic; discrete/continuous.

The environment is single-agent, fully observable, deterministic and discrete.

- (b) Formulate this problem precisely as a search problem: describe the state space, the initial state, the actions, the transition model, the goal test, and the cost function.

State space: Each state is described by Friendly Yellow Guy's position and the location of the uneaten energy pills.

Initial state: Friendly Yellow Guy is in the position shown in the picture and there is a pill in every position of the grid.

Actions: Move Friendly Yellow Guy to an adjacent position.

Transition model: When Friendly Yellow Guy moves to a new location, he also eats the energy pill at that location, if there is one.

Goal test: There are no pills in the grid.

Cost function: The cost of each action is 1.

- (c) Describe in your own words or pseudocode the deep-first search algorithm.

Depth-First Search (DFS) is an algorithm that explores a tree as far as possible along each branch before backtracking, thus going “deep” into the tree.

Pseudocode:

```
function DFS(node, goal, visited):
    if node == goal:
        return Success
    visited.add(node)
    for each child in Children(node):
        if child not in visited:
            if DFS(child, goal, visited) == Success:
                return Success
    return Failure
```

Explanation:

- Call the DFS function with node the initial node, and visited an empty list.
- At each node:
 - If the node is the goal, the search succeeds.
 - Otherwise, mark the node as visited and recurse on each unvisited child.
- If no child leads to a solution, the function returns failure and backtracks.

- (d) Assume that the possible moves for Friendly Yellow Guy are always tried in the order Left, Up, Right, Down. Write the position of Friendly Yellow Guy in the nodes generated by depth-first search until the first repeated state is reached.

D3-D2-D1-C1-B1-A1-A2-A1-A2.

Yellow Guy will first try to go left until reaching D1. Since it can not go more left after it will go up until A1. Then he will go right to A2 and after this will go back to A1. The state is still different since the first time it visited A1 the energy pills were different. Then Yellow Guy will go back to A2 in the same state.

- (e) Consider the heuristic that assigns to each state the number of energy pills left. Explain briefly why this heuristic is admissible. With the same ordering of moves as given before and no check for duplicate states, why is greedy best-first search unable to find a solution to the problem?

The heuristic is admissible because eating one pill requires one move, so the number of moves needed to eat the remaining pills is at least the number of pills left.

Friendly Yellow Guy will first eat all the pills in the upper part of the maze except for those at B2 and C2. Next he will eat the pills in the lower part of the maze, and reach position E3. From here all the next states have the same value of the heuristic (2 for the energy pills in B2 and C2), so it will go left and loop back to E3, never reaching the missing two pills.

2 Slippery Day

Consider the following information.

- If it is sunny, then the grass is dry.
- The grass is wet if its raining.
- If the grass is wet and I am wearing sandals, then it is slippery.
- My feet are cold if I am wearing sandals and its raining.
- Today, it is raining and I am wearing sandals.

- (a) Write down logical representations for these sentences, suitable for use with forward and backwards chaining. Use a suggestive vocabulary.

Using the following propositional symbols (note that this is only one example of a logical representation)

Sun: It is sunny.

Dry: The grass is dry.

Wet: The grass is wet.

Rain: It is raining.

Sandals: I am wearing sandals.

Slip: It is slippery.

Cold: My feet are cold.

Sun $\rightarrow$ Dry

Rain $\rightarrow$ Wet

(Wet $\wedge$ Sandals) $\rightarrow$ Slip

(Rain $\wedge$ Sandals) $\rightarrow$ Cold

Rain $\wedge$ Sandals (these can be considered also as two facts Rain and Sandals)

- (b) Apply forward chaining to show that it is slippery today. You only need to show the successful steps – i.e., you can assume that you always choose the most helpful formulas.

(Rain) and (Rain $\rightarrow$ Wet) gives (Wet).

(Wet), (Sandals) and (Wet $\wedge$ Sandals $\rightarrow$ Slip) gives (Slip)

- (c) Describe in your own words or pseudocode the backward chaining algorithm.

Backward chaining is a goal-driven reasoning algorithm. It starts with the query (goal) and works backwards, looking for rules that can infer the goal. If such a rule is found, the algorithm attempts to prove all the premises (subgoals) of that rule. This process continues recursively until it reaches known facts (base cases), or until it can no longer find applicable rules.

Pseudocode:

```

function BackwardChaining(Rules, Goal):
    if Goal is a known fact in Rules:
        return True
    for each rule in Rules where Conclusion <- Premises:
        if Conclusion = Goal:
            if all BackwardChaining(Rules, premise) for premise in Premises:
                return True
    return False

```

Explanation:

- If the goal is already known (i.e., a fact), we are done.
- Otherwise, we look for rules that conclude the goal.
- For each such rule, we attempt to recursively prove each of its premises.
- If all premises of a rule can be proven, then the goal is proven.
- If no rule leads to a successful proof, the goal cannot be established.

- (d) Apply backward chaining to show that it is slippery. You only need to show the successful steps – i.e., you can assume that you always choose the most helpful formulas.

Slip ->

Wet, Sandals ->

Wet ->

Rain ->

->

- (e) Convert your logical representation to conjunctive normal form (CNF) and use resolution to show that it is slippery today.

-Sun $\vee$ Dry

-Rain $\vee$ Wet

-Wet $\vee$ -Sandals $\vee$ Slip

-Rain $\vee$ -Sandals $\vee$ Cold

Rain

Sandals

We add -Slip to the clauses above try to derive the empty clause []

(-Rain $\vee$ Wet) and (Rain) gives (Wet)

(-Wet $\vee$ -Sandals $\vee$ Slip) and (Wet) gives (-Sandals $\vee$ Slip)

(-Sandals $\vee$ Slip) and (Sandal) gives (Slip)

(-Slip) and (Slip) gives []